

BIC Coursework Report

PSO-ANN for concrete compressive strength prediction

Arne Jacobs – H00508832 – aj4023@hw.ac.uk
Antoine Esman – H00510415 – ave4000@hw.ac.uk

I. Introduction

This project is centred around the prediction of compressive strength of concrete with an ANN. As this is a nonlinear regression problem influenced by several material and mix design factors Artificial Neural Networks are well suited for this task. However, their performance depends heavily on choosing good weight values and hyperparameters. Thus, instead of using gradient-based training, this project applies Particle Swarm Optimisation (PSO) to optimise the ANN's weights.

In this coursework we need to study how different PSO hyperparameters—such as inertia, cognitive and social coefficients, informant strategies, swarm size, and velocity limits—affect ANN training performance. Therefore, we are first required to implement both the ANN and PSO.

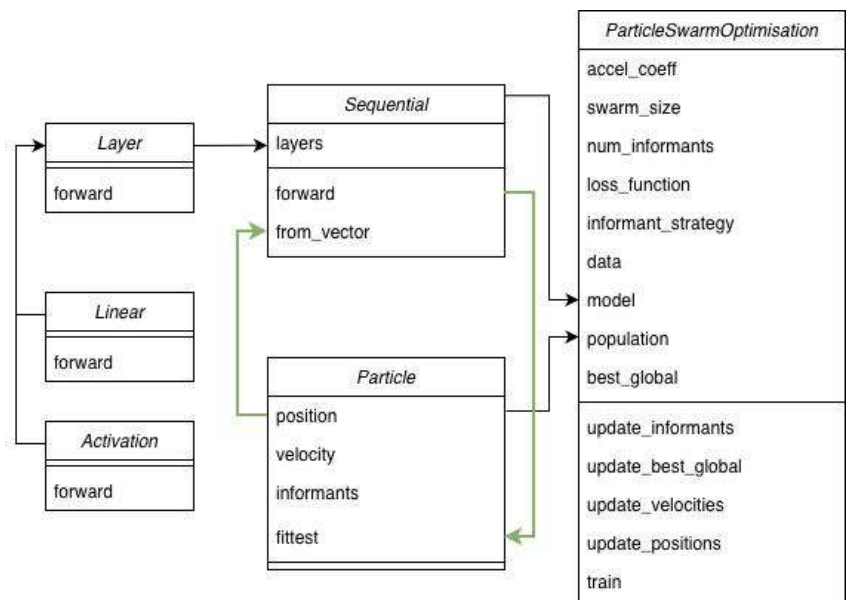
Afterwards, to further improve optimisation, we decided to use a Genetic Algorithm to search for effective PSO configurations. GAs are time-consuming to run thus we also developed a simple web interface to allow 24/7 online running of the GA.

This combined approach provides insight into how biologically inspired optimisation methods can improve ANN-based prediction of concrete strength.

II. Implementation

a. ANN and PSO

On a *Jupyter Notebook*, we used different cells and classes for different parts of the model enabling us to ensure their functionality. As the notebook became larger, we split the classes into multiple python files for easier manipulation. The model evolves around three main files: *data.py* which loads, prepares and normalises the data, *sequential.py* that creates the ANN and *pso.py* that combines both and implements Particle Swarm Optimisation.



(For a more detailed Class diagram see Appendix A)

To create a modern and modular model for our ANN, we coded a *Sequential* class that takes multiple layers. Each *Layer* can either be *Linear* (a hidden layer) or *Activation*, this easily allows to create models with various topologies. Stacking layer on layer also simplifies the feed forward through the ANN.

The main algorithm is in *ParticleSwarmOptimisation*, which contains the list of particles (*Particle* class), the data (X and Y), the hyperparameters as well as the *Sequential* model we want to use. We treat each particle's position (the ANN's weights & biases) as a vector to easily apply its velocity when updating. Then, the particle's fitness is calculated thanks to the *from_vector* and *forward* methods in the *Sequential* model, reusing it for memory efficiency.

Each particle is assigned its own position, velocity, fitness and informants. It also contains the methods for updating its position and velocity. At initialisation, the PSO class creates a population of particles. It can train itself for a set number of epochs and returns the best ANN (fitness, w&b) with the losses over epochs.

In theory, this structure allows us to work with other types of ANN architectures, as long as it implements *forward*, *from_vector*, *to_vector* and *randomize* methods.

Additionally, PSO uses an error metric from `utils.py` to calculate the losses, and more importantly, the `InformantStrategy` (random, k-nearest) that defines how the PSO will select informants for each particle.

Finally, there is an accuracy function, `get_accuracy`, which evaluates our results through percentages.

Until this point, everything was coded by hand sitting and working together on the implementation of both ANN and PSO. This way we both understood the whole code which enabled smoother progression.

In the above, we went further by implementing a sequential model and the k-nearest informant strategy

b. GA

As both of us had been eager to try out Evolutionary Algorithms we decided to implement a Genetic Algorithm to search for the optimal PSO hyperparameters. However, as this is not the focus of the project and just a means for an end, we leveraged the power of LLM (ChatGPT) to help us build it. *GA* contains *PsoGenome* that takes the PSO's hyperparameters. *PsoGenome* has methods to apply mutations and crossovers on them. Additionally, there is *PsoEvaluator* that tests all the PSOs (3 times to minimise lucky runs) and returns their `avg_accuracy` in a cache. The whole GA is orchestrated by the `GeneticPsoOptimizer` which runs multiple generations of a certain population where it selects some elites and some individuals by tournament.

c. Web Interface

The GA needs a lot of time to run so we implemented a web interface with *Streamlit* to enable running our code in background on a server to search for optimal PSO hyperparameters. This web interface is very basic and was mainly implemented by ChatGPT's as it was just a tool for visualising and online training. Still, it couldn't get it right quickly and we had to guide it a lot. The implementation is made possible through `train_manager.py` that runs the GA, `db.py` that saves the data and `app.py` that creates the web interface.

d. Parallel running

Even running in background, it took days to complete these GA runs and we thought of multi-threading the population of the GA. Unfortunately, it was conflicting with the database, so we removed it. However, to run experiments later, we parallelised the runs of genomes and it made our life easier.

e. GA animation

Because it's always nice to visualise, we can plot the position of the particles on a 2d map and animate it over epochs. To reduce the dimensions (from hundreds or thousands to 2), we evaluated different techniques, such as the PCA, UMAP and t-SNE. The PCA gave the best results for our use-case and showed convergence of particles during training. This bonus was also assisted by ChatGPT, but the research and the methodology was done by hand. [5]

III. Results and discussion

We used “experiments.ipynb” to run all our experiments and started off with the hyper-parameters of the best PSO configuration we found through our GA web interface.

1. GA and the base configuration

Based on some literature and recommendations found online we made a base configuration for testing hyperparameters (see Appendix B page 10) [1][2][3][4]. It basically is a set of hyperparameters which we try to keep the same for similar tests except for the hyperparameter that we’ll be testing. As we did some testing with these hyperparameters we quickly noticed that the results were way less interesting than with the optimised PSO that the GA had given us. Therefore, we decided to use the GA’s best PSO configuration instead.

For topology testing	For after topology testing
Iter = 20 Topology = [8, 4] Activation function ReLU Small weight initialisation in (0.0001, 0.05) Swarm = 45 Epochs = 200 Inertia = 0.708 Cognitive = 1.898 Social = 0.351 Global Best = 0.684 J = 0.851 N = 4 K-nearest Boundary reset Max p and v = 3.87, 0.9 End on number of iterations MSE	Iter = 20 [8, 4] ReLU Small weight initialisation in (0.036145, 0.0893) Swarm = 45 Epochs = 200 I = 0.708 C = 1.906 S = 0.288 GB = 1.3 J = 0.76 N = 6 K-nearest Boundary reset Max p and v = 3.87, 0.9 End on number of iterations MSE

The second configuration comes from a new GA run that gave us better results than before.

2. Hyperparameter of the ANN

a. Topology (Annexe C page 11)

Topology	Time	Avg_train	Avg_test	Std_train	Std_test
Perceptron ReLU	0m48	76.54	77.66	0.01	0.04
Perceptron no ReLU	0m47	76.53	77.66	0.02	0.02
[12] no ReLU	2m11	76.54	77.66	0.01	0.02
[8, 4] no ReLU	2m05	76.57	77.67	0.01	0.02
[8, 4]	2m09	84.09	84.66	1.54	1.69
[2]	1m02	75.20	76.45	4.62	4.02
[6]	1m58	78.37	79.32	3.01	2.87
[8]	2m26	81.92	82.51	2.81	2.76
[12]	2m56	80.80	81.51	2.59	2.45
[12]	4m06	81.38	81.99	2.85	2.75
[8, 4]	3m55	82.90	83.48	2.13	2.18

Light blue: running with 100 iter for checking best

Dark blue: running with swarm size = 60, epochs = 500, iter = 20 (forgot to change back)

By running the different topologies, we had some interesting observations:

- Although the Perceptron is very fast to train, it still has a decent accuracy.
- When running different topologies without ReLU layers, the final accuracies are always almost the same and the stds are almost 0. We found this very intriguing and figured this is due to learning issues without ReLU showing its necessity for training.[\[2\]](#)
- Out of the single layer models, it seems that the one with 8 nodes performs the best. (Topology [2] seems very unstable). Similarly, in the two-layer models, the ones starting with a first layer of 8 seem to outperform the others.
- It looks like [8, 4] has the best compromise between training time and final accuracy among topology among those tested. This perfectly adds up with what we had found in the literature where some recommended to have the first layer size between in and output sizes. It also recommends having a second layer to get more complexity without having one very big layer size.[\[1\]](#) Thus this is the structure that we will use for the PSO's hyperparameters experiments.
- The 3-layer topologies did not perform that well ($\text{accu}([8, 4, 2]) < \text{accu}([12])$ but same duration). We imagine that 3 hidden layers is too much for this problem.
- Topologies with the same number of weights and biases (e.g. [12] and [8, 4]) tend to have a similar runtime which is expected. However, we notice that single layer topologies are slightly slower (4-11%).

3. PSO

a. Swarm sizes and epochs (Appendix D page 12)

Swarm_size	Time	Avg_train	Avg_test	Std_train	Std_test	Epochs
20	15.7	77.11	77.97	3.12	2.99	100
20	22.7	80.83	81.48	2.8	2.71	200
20	28.5	81.00	81.63	2.66	2.48	300
20	37.0	81.34	82.05	2.72	2.5	400
100	2.20.7	84.69	85.34	0.52	0.64	300
100	3.08.0	84.78	85.42	0.86	0.89	400
100	3.49.5	84.47	85.15	1.85	1.89	500

We ran different swarm sizes while looking at the “Loss over epochs” to estimate number of epochs. We notice that the accuracies get better as the swarm size increases. This is due to the higher number of individuals enabling vaster exploration. The tests also show that when the swarm size increases, we often need to increase the number of epochs to converge, thus increasing accuracy. However, running too many epochs (e.g. 100 individuals with 500 or 600 epochs) shows no improvement because it reached an optima earlier (e.g. 400 epochs). After all these runs we decided to keep a swarm size of 20 with 300 epochs for fast and decent experimentations on the other hyperparameters.

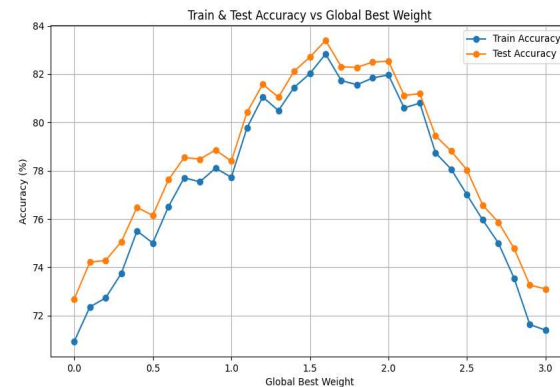
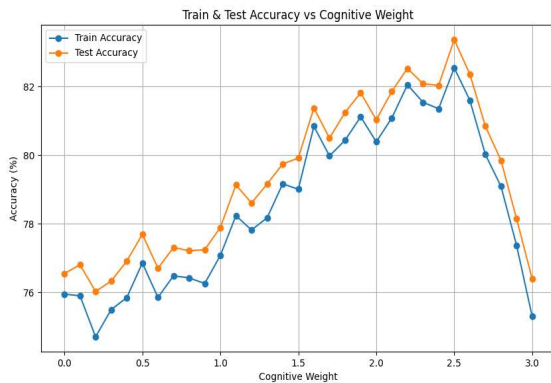
Swarm_size	Time	Avg_train	Avg_test	Std_train	Std_test	Epochs
25	28.4	82.11	82.78	2.62	2.57	200
200	39.7	75.14	75.99	1.97	1.91	25
50	1.51.4	84.13	84.80	0.99	1.16	500
500	4.31.7	80.17	80.69	2.17	2.05	50

These tests show different ways of allocating solution evaluations.

Having more epochs clearly gives better accuracy (4-7%) and runtime (25-40%). Completely training some individuals outperforms tons of less trained individuals. However, a balance is required to avoid extreme cases such as low swarm size converging to local minimum (e.g. 25s/200e outperforms 10s/500e).

b. Acceleration Coefficients all from 0 to 3 with an incrementation of 0.1

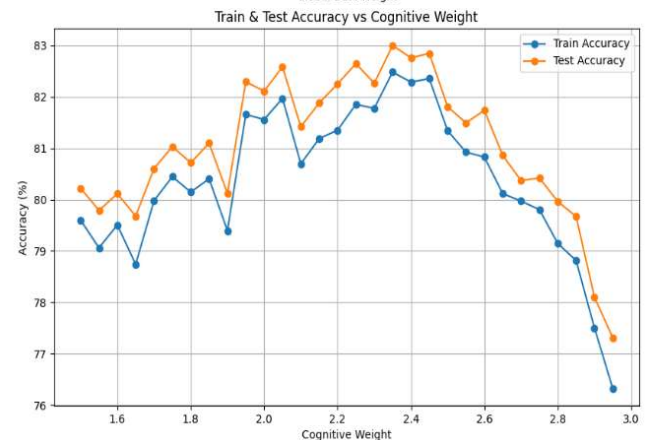
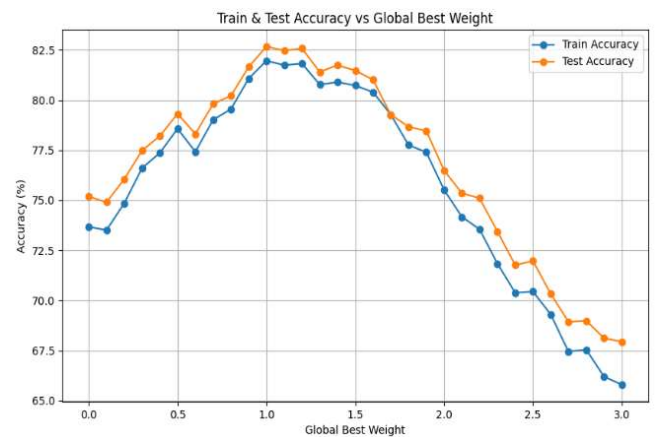
(Appendix E page 13)



After running the 20 iterations of the PSO for every incrementation of any acceleration coefficient we notice that our GA gave us decent values but that they are not perfect. Inertia seems perfect since the highest accuracy is at inertia = 1.7 which is the same as our PSO. However, for the three other coefficients this is not the case ($C = 1.906 < 2.5$, $S = 0.288 < 0.8$, $GB = 1.3 < 1.6$).

We thought running the global best variation again, with a cognitive weight of 2.5 instead of 1.9, would increase accuracy. Instead, the overall accuracy of the global best variation decreased. We decided to run the variation again to check the reliability of the plot and we got a very similar result. Thus there is a high chance that $C = 2.5$ is not really an optimal cognitive weight in our case.

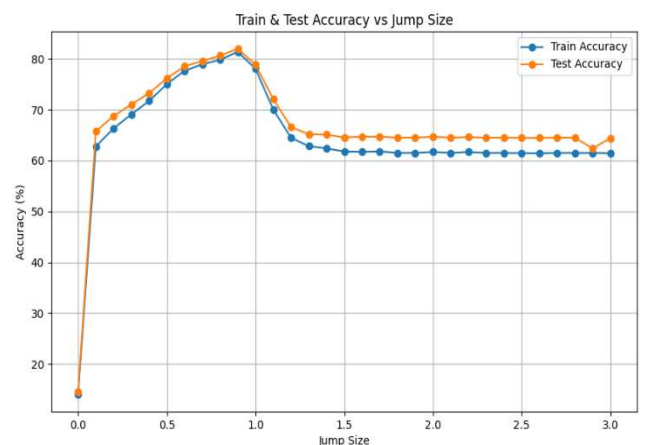
To test this, we run the cognitive weight plot again but more focused from 1.5 to 3.0. The plot is quite close to the previous one, but the peaks are in different positions showing a lack of reliability. We then plot again from 2.3 to 2.8 and it is still unreliable. Adding more iteration would increase reliability, but we could also use the box-and-whiskers technique to better analyse the results.



c. Jump size

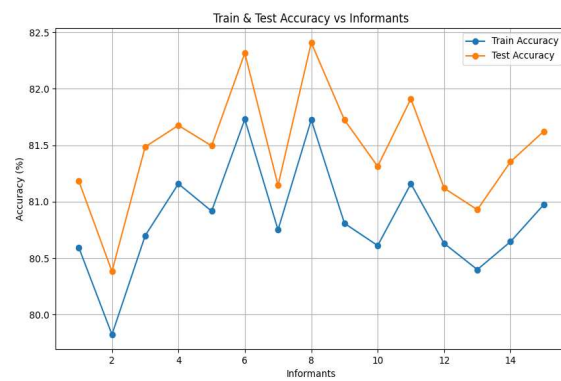
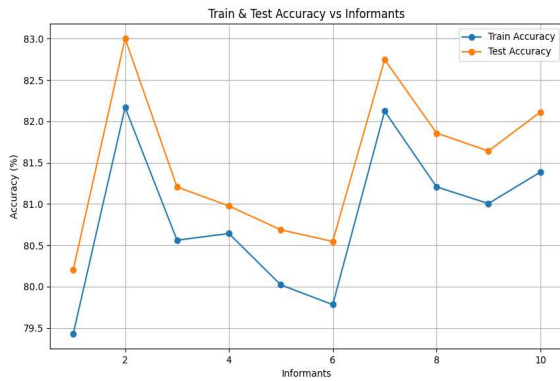
(Appendix F page 14)

We notice that on two identical runs we have almost identical results so the jump size is trustworthy. The best value seems to be near 0.9 and the accuracy plateaus after 1.3. We think this plateau may be due to our velocity restrictions.



d. Informants

i. Number



We notice that the accuracy given by the number of informants is completely unreliable and almost chaotic. We think that this is the reason why our GA always gave us optimised PSO configurations with low or even 0 values for social weight. Our other parameters might be too “aggressive” so the particles move around too much and become unreliable informants for other particles.

ii. Random vs K-nearest

Topology	Time	Avg_train	Avg_test	Std_train	Std_test
K-Nearest	3m55	82.90	83.48	2.13	2.18
Random	3m19	83.02	83.68	2.28	2.24

Both strategies yield similar results which it not surprising, considering our low social weight and our previous findings about informants.

4. Conclusion

Creating an artificial neural network and particle swarm optimisation from scratch was challenging, but very educative. PSO does not rely on derivatives (like backpropagation) and therefore avoids issues like vanishing gradients. Additionally, it performs better in presence of multiple local minima.

Our research led us to explore different hyperparameters for both ANN and PSO. We made a genetic algorithm trained to find the best PSO configuration and used its results as base for the experimenting. Considering its outputs and our research, the best topology we found to solve the concrete compressive strength problem is to have 2 hidden layers with 8 and 4 neurons respectively. Moreover, the acceleration coefficients provided by the GA are fairly close to the best values our experiments gave us.

With more time, we would have loved to add more visualisation and control to the web interface to better understand the fine tuning of PSO hyperparameters. We would have also manually tested more PSO configurations that evolve around informants and social weight.

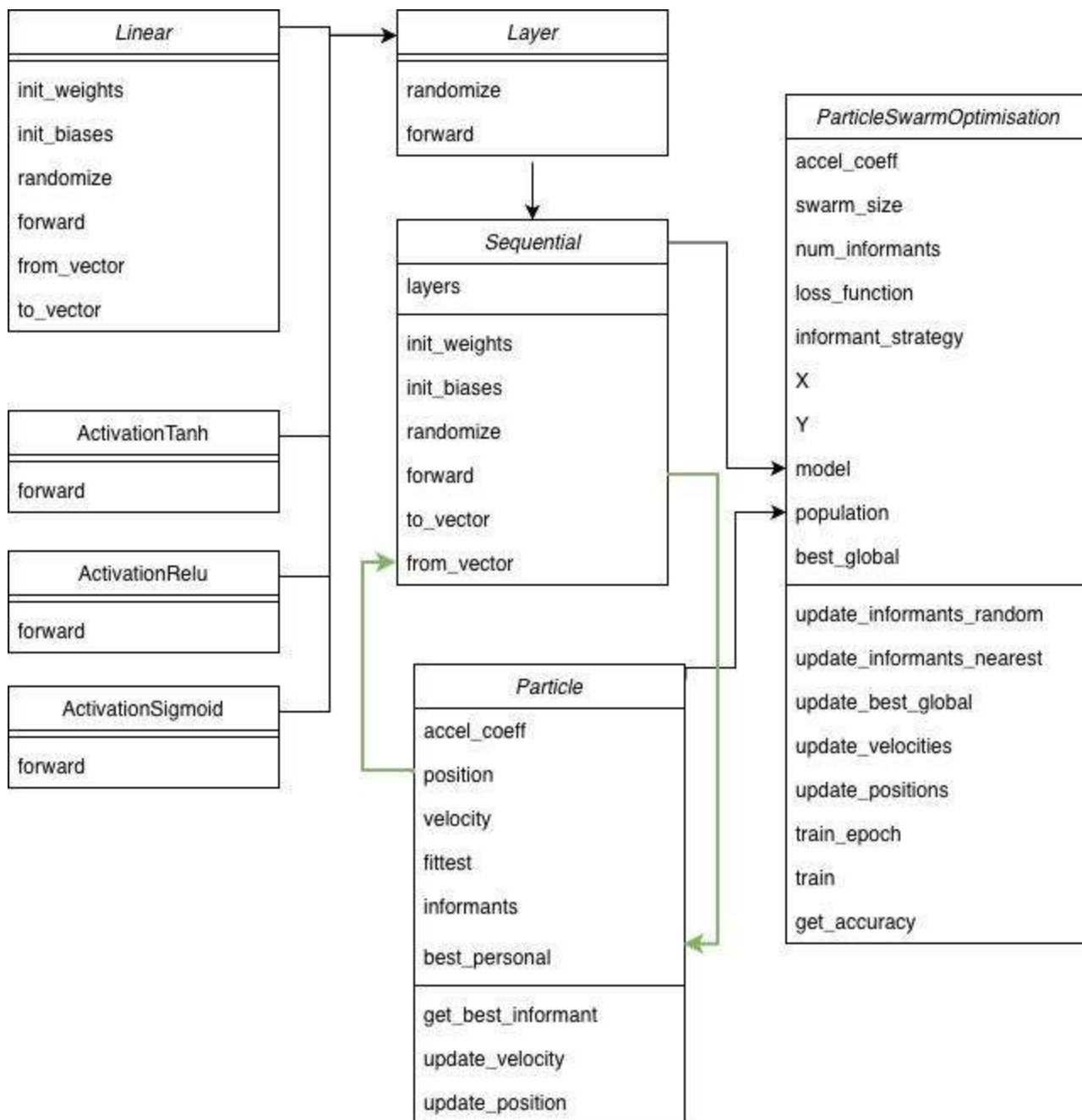
To get even better results, many state-of-the-art methods combine PSO for exploration and backpropagation on the ANN for local fine tuning.

5. References

- [1] Heaton, J. (2008). *“Introduction to Neural Networks for Java.” Chapter 5.*
- [2] Goodfellow, Bengio & Courville (2016). *“Deep Learning,” Chapter 6.3*
- [3] Clerc & Kennedy (2002). *“The Particle Swarm—Explosion, Stability, and Convergence in a Multidimensional Complex Space.”*
- [4] Kennedy & Mendes (2002). *“Population Structure and Particle Swarm Performance.”*
- [5] Deeipa (2024). *“Latent Space Visualisation: PCA, t-SNE, UMAP”*
https://www.youtube.com/watch?v=o_cAOa5fMhE

Appendices

A. Class diagram (ANN and PSO)



B. Selection of base hyperparameters for experimentations

Literature on neural network design recommends choosing hidden layer sizes between the size of the input and output layers, rather than excessively large architectures, to avoid overfitting and unnecessary computational cost. [1] Also, adding a second layer can easily increase complexity and speed instead of one big layer. We thus chose [IN, 8, 4, OUT]. This structure also experimentally proved to give the best balance between speed and accuracy in our cases.

We use the **Rectified Linear Unit (ReLU)** activation function between hidden layers. ReLU is widely regarded as the default choice for modern regression networks because it avoids the saturation and vanishing-gradient characteristics of sigmoid/tanh, and creates a piecewise-linear model that is easier for evolutionary and swarm optimisers to navigate. [2]

Based on the GA's return and the knowledge that large weight initialisations often cause divergence or exploding forward functions, we decided to have w in ± 0.0001 and b in ± 0.05 .

Online it seemed that a swarm size between 30 and 50 is recommended to balance exploration and computational cost thus we chose a swarm size of 40.

Our first runs showed us that the number of epochs needed to stabilise could greatly depend on luck and other hyperparameters. We decided to start with 100 epochs and check out the loss plots to see if it mainly finished converging. If not, we would increase epochs and if the run seems to always finish before the last epoch we would decrease the number.

For our acceleration weights we decided to follow the recommendations of the constriction factor PSO variant: inertia = 0.729 and cognitive = social = 1.49618. These values ensure stable convergence by controlling exploration and preventing divergence of velocities. [3]

The weight given to the best individual's position is not usually a parameter in the PSO. To use the best's position while still listening well to inertia, social and cognitive we decided to put a mid-low value (0.5).

We use a small jump_size of 0.1 to avoid the population to expand too quickly.

Sources say that the number of informants should always be between 10 and 25% of the swarm size. [4] We noticed that the GA mainly gave us low numbers of informants and low social weights which made us think that informants may not be the best option for this problem. Thus, we decided the lowest values of 10% i.e. 4. The 4 informants would be selected every iteration as the four closest particles to the trained particle.

We have only implemented position reset when going out of boundaries so we will use that one. The boundaries we define are both on position (3.87) and velocity (0.9) and the values come from the GA.

Otherwise, we will finish a run when it finishes a certain number of iterations, and the errors will be measured in MSE as we seemed to understand that this is the standard one for continuous regression targets.

C. All topology results

Topology	Time	Avg_train	Avg_test	Std_train	Std_test
Perceptron ReLU	0m48	76.54	77.66	0.01	0.04
Percept no ReLU	0m47	76.53	77.66	0.02	0.02
[2]	???	76.85	77.93	1.34	1.29
[2]	1m02	75.20	76.45	4.62	4.02
[2]	1m06	77.37	78.34	4.63	4.18
[6]	1m58	78.37	79.32	3.01	2.87
[8]	2m26	82.12	82.69	2.76	2.70
[8]	2m25	81.71	82.33	2.86	2.82
[8]	0m37	80.85	81.64	2.92	2.79
[8]	3m10	81.04	81.73	3.13	3.02
[12]	2m56	80.80	81.51	2.59	2.45
[12]	0m47	80.51	81.27	3.01	2.84
[12]	4m06	81.38	81.99	2.85	2.75
[12] no ReLU	2m11	76.54	77.66	0.01	0.02
[4, 2]	1m38	78.74	79.43	2.53	2.45
[4, 2]	1m41	78.50	79.33	2.72	2.59
[8, 4]	2m48	79.41	80.07	2.95	2.75
[8, 4]	0m42	82.55	83.33	2.71	2.63
[8, 4]	3m55	82.90	83.48	2.13	2.18
[8, 4] no ReLU	2m05	76.57	77.67	0.01	0.02
[8, 4]	2m09	84.09	84.66	1.54	1.69
[12, 6]	4m24	81.14	81.83	2.34	2.41
[16, 8]	6m32	80.68	81.29	2.32	2.45
[32, 16]	16m18	81.53	82.04	1.78	1.77
[24]	5m11	81.65	82.31	2.29	2.20
[48]	9m43	82.32	82.83	1.90	1.97
[48]	15m29	79.64	80.16	2.13	2.21
[8, 4, 2]	3m05	79.48	80.60	2.46	2.36
[16, 8, 4]	6m53	80.45	81.07	1.96	1.80

Yellow: first thread running with new pso configuration

Light blue: running with 100 iter for checking best

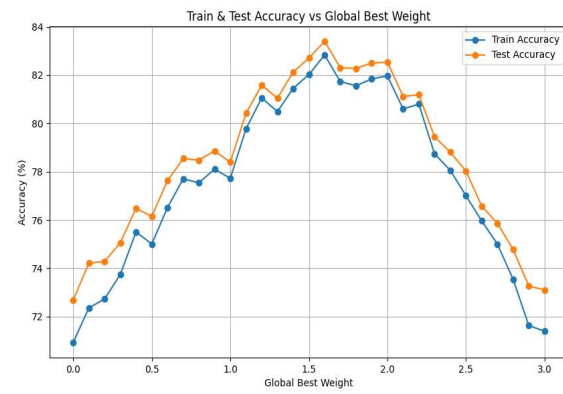
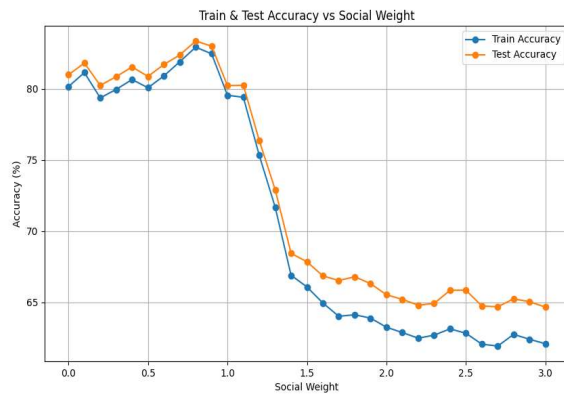
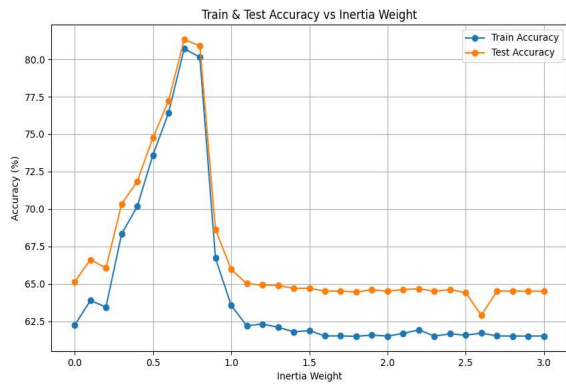
Dark blue: running with swarm size = 60, epochs = 500, iter = 20 (forgot to change back)

D. Swarm sizes and epochs

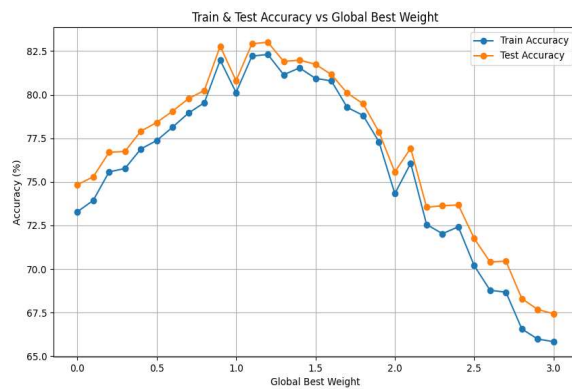
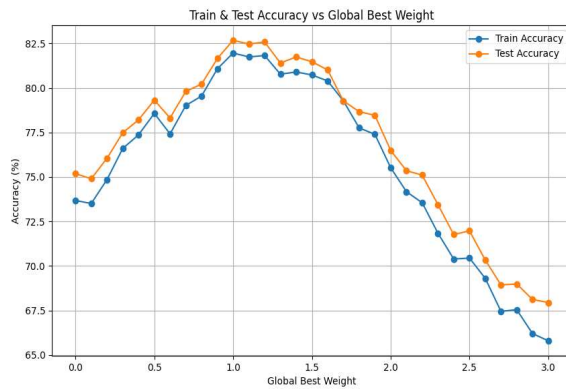
Swarm_size	Time	Avg_train	Avg_test	Std_train	Std_test	Epochs
10	12.4s	74.8	76.32	1.98	1.91	100
10	16.5	77.21	78.04	3.39	3.08	200
10	18.9	76.61	77.50	2.18	1.88	300
20	15.7	77.11	77.97	3.12	2.99	100
20	22.7	80.83	81.48	2.8	2.71	200
20	28.5	81.00	81.63	2.66	2.48	300
20	37.0	81.34	82.05	2.72	2.5	400
30	40.8	81.74	82.26	2.75	2.59	300
30	51.4	82.6	83.15	2.28	2.17	400
45	58.4	82.53	83.28	3.00	2.79	300
45	1.17.9	83.04	83.68	2.69	2.51	400
60	1.20.7	83.78	84.34	1.97	1.94	300
60	1.43.2	84.25	84.93	0.8	0.91	400
60	2.19.1	84.24	84.84	0.93	1.09	500
80	1.50.8	83.56	84.15	2.45	2.41	300
80	2.24.6	83.92	84.62	2.45	2.37	400
80	2.57.6	84.35	85.13	1.99	1.97	500
100	1.41.3	83.97	84.74	2.01	2.05	200
100	2.20.7	84.69	85.34	0.52	0.64	300
100	3.08.0	84.78	85.42	0.86	0.89	400
100	3.49.5	84.47	85.15	1.85	1.89	500
100	4.52.9	84.73	85.35	0.66	0.81	600
Comparing						
10	10.9s	70.52	71.65	2.25	2.40	50
50	12.0	67.56	68.88	2.67	2.86	10
20	16.3	77.24	78.34	2.53	2.33	100
100	18.1	72.39	73.46	1.81	1.92	20
40	17.3	75.53	76.76	2.49	2.06	50
50	17.1	74.76	75.80	1.84	1.29	40
50	28.5	79.66	80.44	3.14	3.19	100
100	32.1	77.13	77.77	1.97	2.08	50
25	28.4	82.11	82.78	2.62	2.57	200
200	39.7	75.14	75.99	1.97	1.91	25
10	26.2	78.18	79.07	3.49	2.97	500
500	59.9	73.20	73.24	1.48	1.45	10
100	2.05.2	83.93	84.56	1.80	1.77	250
250	2.58.0	82.65	83.23	2.16	2.27	100
50	1.51.4	84.13	84.80	0.99	1.16	500
500	4.31.7	80.17	80.69	2.17	2.05	50

E. Acceleration Coefficients

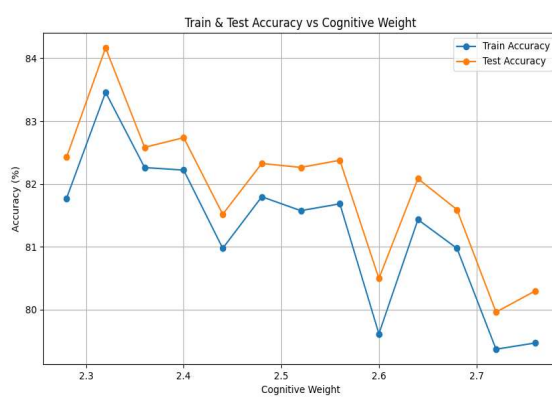
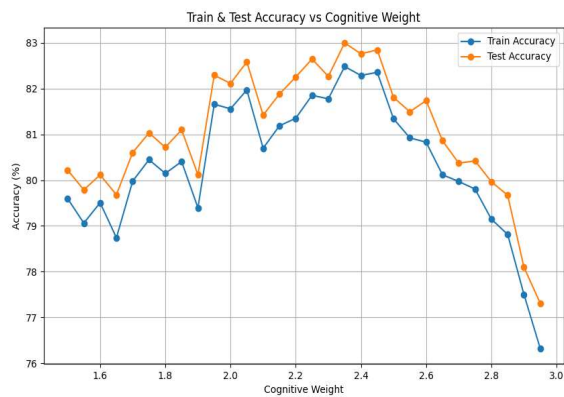
a. Checking the different plots



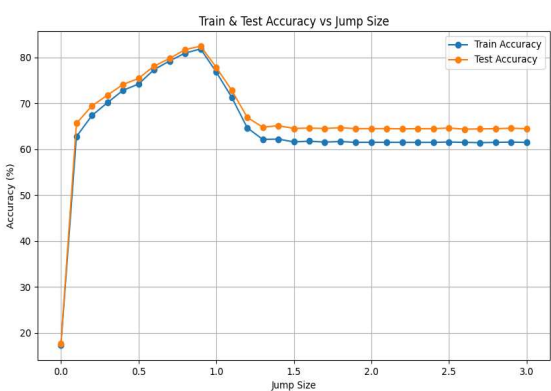
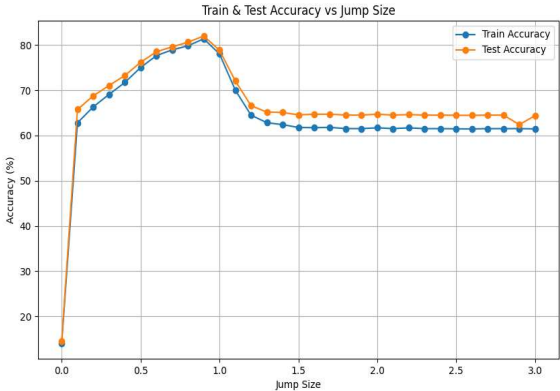
b. Checking the influence of modifying the cognitive weight



c. Zooming in on the cognitive weight plot

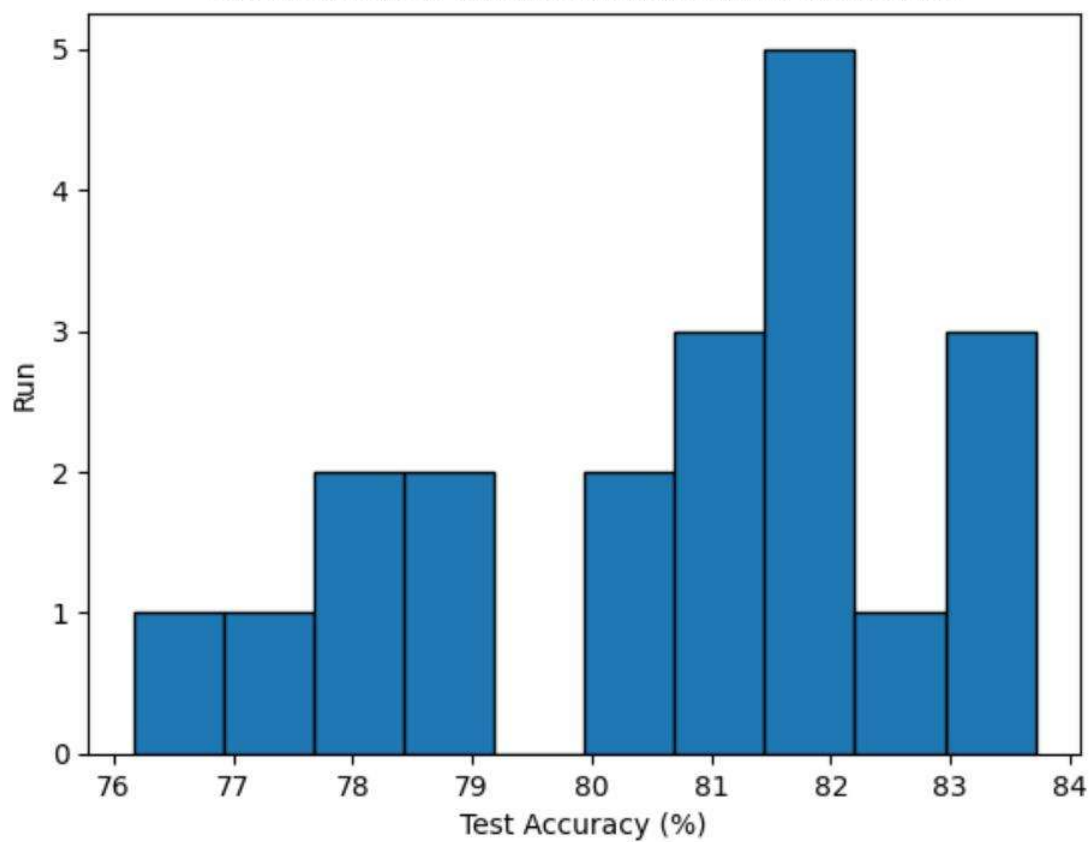


F. Jump size

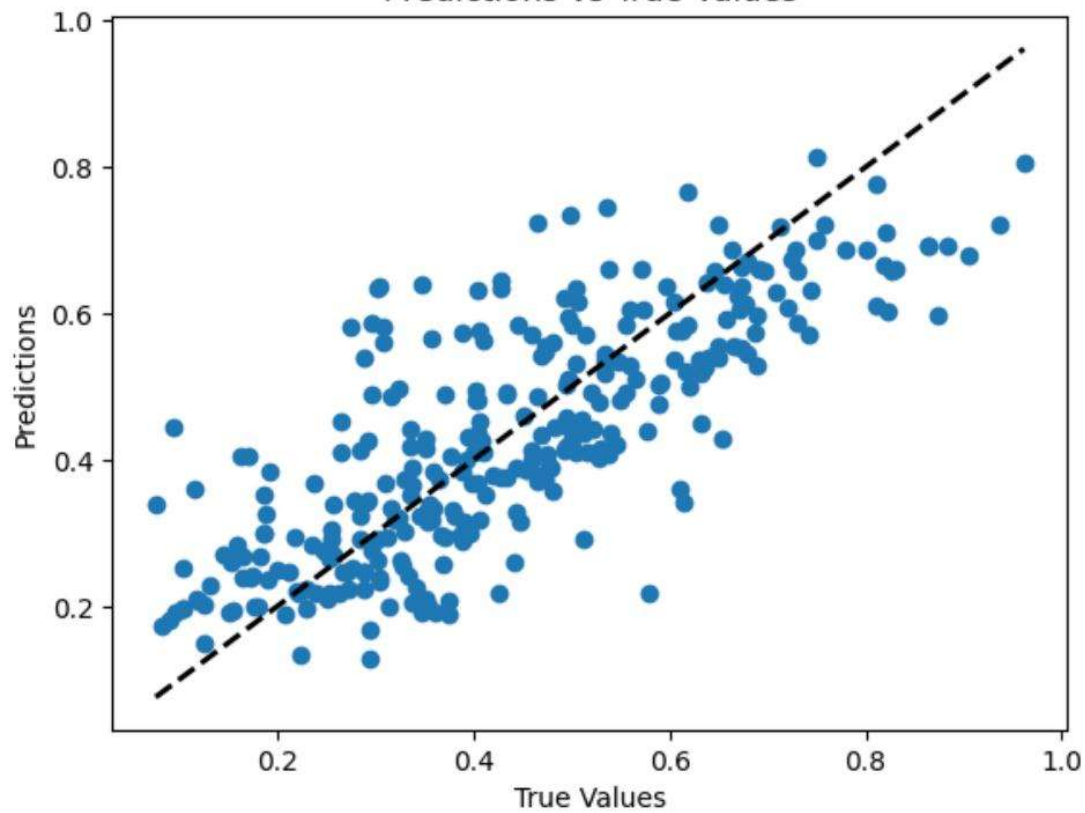


G. Others

Distribution of Test Accuracies over PSO Runs



Predictions vs True Values



Training Loss over Epochs

